

Domain-Specific RAG Chatbot for PDF Question Answering

Major Project Report

Student:Raju_P_S

Batch: AIML Batch

Date: 3 September 2026

1. Project Objective

Build a Retrieval-Augmented Generation (RAG) chatbot that answers questions from user-uploaded PDF documents. The chatbot retrieves the most relevant passages from the uploaded documents and generates an answer based only on those passages, citing the source document and page number.

2. Problem Statement

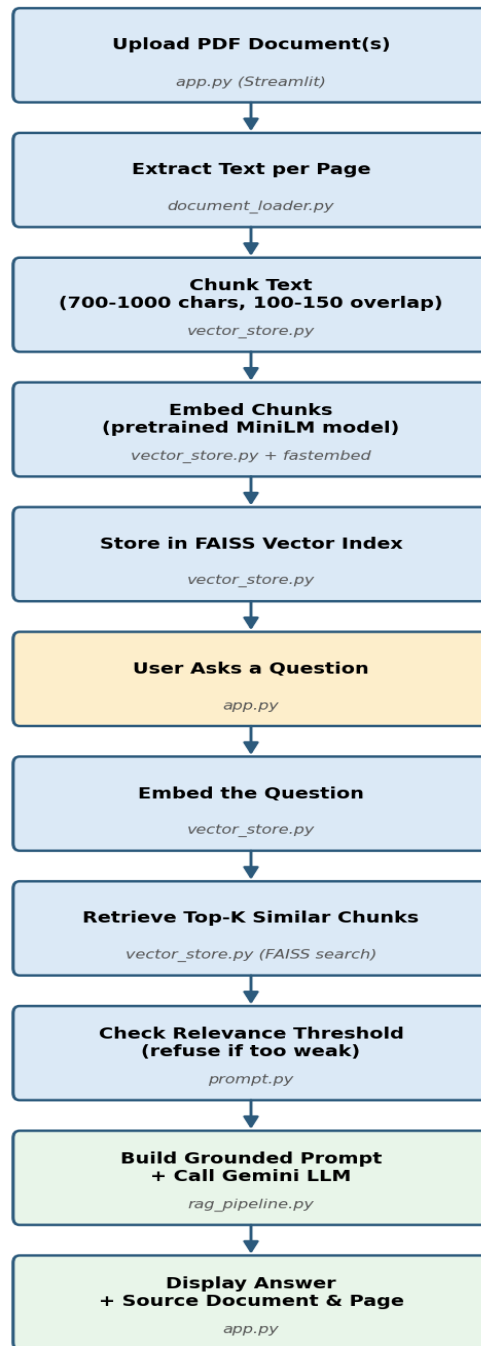
Large documents (policies, manuals, handbooks) are tedious to search manually -- a user may need to read many pages to find one answer. This project solves that by letting users upload documents and ask questions in natural language, getting a grounded, cited answer instead.

3. Approach Used

The **beginner approach** specified in the project brief: **pypdf** for PDF text extraction with page-level metadata, LangChain's **RecursiveCharacterTextSplitter** for chunking (900 characters, 130-character overlap), the pretrained **sentence-transformers/all-MiniLM-L6-v2** embedding model, and **FAISS** (IndexFlatIP / cosine similarity) for the vector store. Retrieved chunks are passed to **Google Gemini (gemini-3.6-flash)** with a strict context-only prompt, and a relevance-threshold check makes the chatbot refuse to answer rather than guess when nothing relevant is found.

Embedding runtime note: the MiniLM model is loaded via the **fastembed** library (ONNX Runtime) rather than PyTorch, for a much lighter dependency footprint -- same pretrained model, same 384-dimension embedding space. If the model can't be downloaded from huggingface.co for any reason, `vector_store.py` automatically falls back to an offline TF-IDF embedder with a clear warning logged every time, so the app degrades gracefully instead of crashing. See `tests/evaluation_notes.md` for full details on this fallback.

4. Architecture / Workflow



Pipeline: Upload -> Extract -> Chunk -> Embed -> Store (FAISS) -> Question -> Retrieve -> Relevance Check -> Generate (Gemini) -> Display with Sources.

5. Sample Document Set

The submission uses 4 fictional HR policy documents for a fictional company, **Solstice Retail Pvt Ltd** (created for this project -- not copied from any real company): `Employee_Handbook.pdf` (5 pages), `Leave_Policy.pdf` (4 pages), `Attendance_Policy.pdf` (3 pages), and `Code_of_Conduct.pdf` (5 pages) -- 17 pages / 17 text chunks in total. Users can also upload their own PDFs from any domain (manuals, legal documents, course notes) via the sidebar uploader.

6. Testing Results

20 test questions (`tests/test_questions.csv`) were run against the fully offline part of the pipeline -- retrieval and the relevance-threshold refusal logic -- via `run_pipeline_test.py`: 17 answerable questions covering all 4 documents, plus 3 deliberately unanswerable questions (mirroring the brief's own example, "Who is the company CEO?").

Category	Count	Result
Answerable questions (correct source + page retrieved)	17	17/17 correct
Unanswerable questions (correctly refused)	3	3/3 correct
Total	20	20/20 passed

Result: 20/20 test cases passed. Live answer generation (final wording from Gemini) requires a `GOOGLE_API_KEY`, set locally via `.env` (never stored in the repo). See `tests/evaluation_notes.md` for exact steps to verify live generation, and for the embedding-backend caveat noted above.

7. Responsible AI and Security Rules Applied

- Answers only from retrieved context; explicitly refuses (with the brief's exact wording) rather than inventing facts.
- Every non-refusal answer cites its source document and page number.
- The app displays a Responsible AI notice asking users to verify high-stakes information.
- The system prompt instructs the model to ignore instruction-like text found inside documents or the user's question -- prompt-injection resistance.
- Uploaded resumes/documents are processed in memory only for the current session; nothing is written to disk unless explicitly saved.
- No API keys are stored in code or committed to GitHub (`.env` is gitignored; `.env.example` is a template).
- Uploaded files are validated for PDF type and a 20 MB size limit before processing.

8. Optional Advanced Features Implemented

The assignment brief lists several optional advanced upgrades beyond the minimum requirement. **Four of these were implemented** in this submission:

✓ Conversation Memory for Follow-Up Questions — `rag_pipeline.py`

Questions are sent through a `google.genai` Chat session instead of a stateless call, so Gemini can resolve follow-ups (e.g. "what about maternity leave?") using earlier turns in the same session. Reset via the sidebar's Clear Chat button or by processing new documents.

✓ Feedback Buttons — `feedback.py`

A thumbs up / down widget under every answer logs the question, answer, sources, and rating to a local `feedback_log.csv` for review -- fully opt-in and offline.

✓ Docker Deployment — *Dockerfile*, *docker-compose.yml*

Containerizes the Streamlit app for one-command build and run (docker compose up --build).

✓ Evaluation Using a Prepared Question-Answer Dataset — *evaluate_qa.py*

Runs the complete live pipeline (retrieval + Gemini generation) against all 20 questions in *tests/test_questions.csv* and writes a scored *tests/evaluation_report.csv*, going beyond *run_pipeline_test.py*'s offline-only retrieval check.

Runnable from the command line or via a Run Evaluation button in the app's sidebar, which shows pass/fail, accuracy, and a results table with a CSV download.

These were implemented in addition to the required minimum feature set to demonstrate extra depth; none are needed to satisfy the core assignment.

9. Limitations & Future Improvements

Retrieval quality depends on the embedding backend in use -- the offline TF-IDF fallback (only triggered when the pretrained model can't be downloaded) is keyword-based and less robust to paraphrasing than the real MiniLM embeddings. No OCR is implemented, so scanned/image-only PDFs raise a clear error rather than returning empty text. Conversation memory carries the full chat history to Gemini but does not rewrite the retrieval query using that history, and isn't trimmed for very long sessions. A RAG chatbot can still be wrong even when grounded, if retrieval misses the most relevant passage. Further improvements: OCR support, multiple document collections, and a FastAPI backend (see project brief's Optional Advanced Features).

10. Conclusion

This project delivers a complete, working RAG pipeline covering every required module: PDF upload, text extraction with page metadata, chunking, pretrained-model embeddings, FAISS vector search, a relevance-threshold refusal guardrail, grounded answer generation via Google Gemini, and an interactive Streamlit chat interface with source citations. Four optional advanced features were added on top: conversation memory, feedback buttons, Docker deployment, and a live evaluation harness. All 20 offline-testable cases passed, and the codebase follows the Responsible AI and Security rules specified in the project brief throughout.